Depth Estimation for Converting Images into Minecraft Schematics

Tyler Mitts
Dept. of Electrical Engineering and
Computer Science,
University of Missouri
Columbia, MO 65211, USA
tmitts0206@gmail.com

***Abstract*—Using a depth estimation pipeline, a 3D reconstruction of 2D images are used to automatically create Minecraft schematic files.**

Keywords—Depth estimation, 3D reconstruction, Segmentation

I. Introduction: Motivation and Background

Programs have been made that can turn images into 2D pixel art in Minecraft, but none have used images to extrapolate 3D information and create 3D builds before. There are also artists within the Minecraft community who create 2D art by using depth (multiple layers of blocks), just like a painter would use layers of paint for more detail. These have led to inspirations of combining the two. With the use of depth estimation, k-means segmentation, and nearest neighbors methods, this was successfully achieved. Builds created from this pipeline can easily be applied to many creative aspects of the game, such as maps for commercial servers, advertisements for specific brands or locations, or even just a cool build to add to a Minecraft world.

II. Image Processing Modules and Pipelines

A. Modules Used

OpenCv, PyTorch, Scikit-Image, NumPy, Matplotlib, OS.

B. Pipeline Used

The input image is first put through the MiDaS depth estimation model, outputting the depth map. This depth map is then segmented using k-means to determine what the z-coordinates should be based on how much depth is in the image. Each cluster from k-means is then segmented into grids with cell size determined by the user, which then each cell is then mapped to a Minecraft block based on its RGB value. The 3D plot of Minecraft blocks are then saved to a Litematica file. This pipeline was selected because it catered to the requirements of the game, depth estimation for general 3D information of image, k-means so that each z-value had integer and accurate values, and grid segmentation so cells can be converted directly into blocks.

III. Approach: Algorithms and Module Implementation Details

Examples from each part of the pipeline will be using figure 1 as the initial input image.

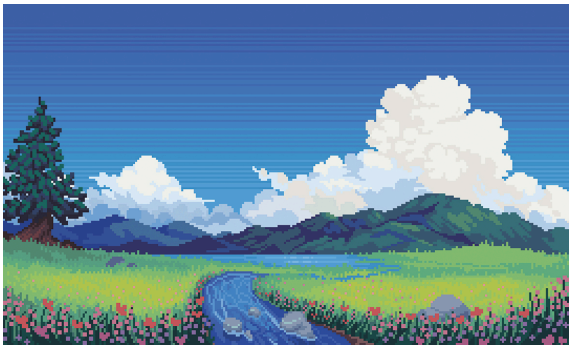


Figure 1. Initial input image.

For depth estimation, the DPT_Large model of MiDaS was implemented using PyTorch [1]. This model was chosen as it is one of the most accurate ones, but comes at the cost of computational speed. Since this application is purely creative and isn't relied on to make important or time sensitive decisions, that isn't a problem. This model is particularly useful as it uses self-attention to extract global features, allowing for more accurate distinctions between depths of subjects, i.e. the global information about the sky.

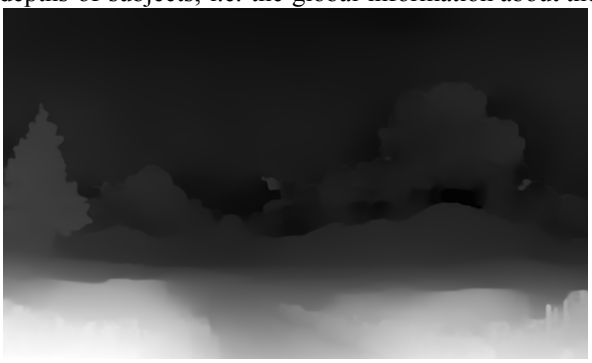


Figure 2. Depth Estimation of input image.

Using the image from figure 2, it is segmented into k number of clusters based on the intensity values.

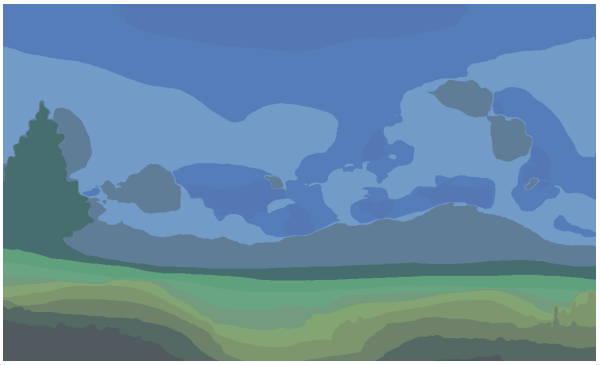


Figure 3. K-means segmentation of depth map.

To determine the k-value used for segmentation, the elbow method was implemented for automating this process [2]. For this function, a max k-value is set at a parameter from the user, and the elbow method will iterate through all k-values up to the max k-value and compute the squared distance between every point in a cluster and its cluster's center for each one. It then determines the "elbow point" of the graph in which the improvement from adding another cluster becomes minimal. The index at this point indicates the optimal k-value, for which is used in k-means for this pipeline.

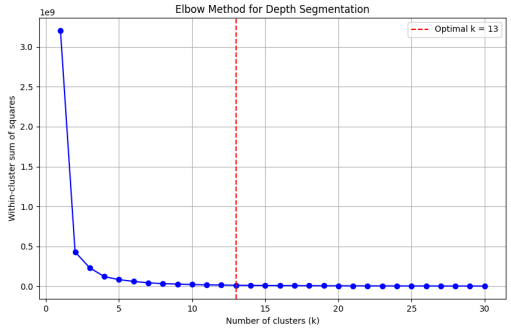


Figure 4. Optimal k-value selection using elbow method.

Each cluster is then segmented into grids, done by finding the boundary box of the cluster, deciding how many rows and columns there should be based on min_cell_size, a parameter set by the user.

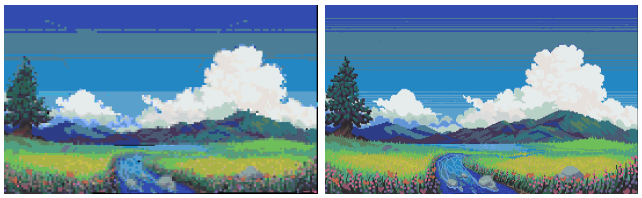


Figure 5. Grid composites with min_cell_size 8 and 2 respectively.

Seen in figure 5, the image that used a larger min_cell_size is much more pixellated and will be much less detailed in the final build, on the other hand though, lowering the min_cell_size increases computing time drastically as it creates many more cells.

For matching each of the cells to different Minecraft blocks, the average color of both the cells and all of the Minecraft textures are averaged, allowing for them to be compared seamlessly. The RGB values of the cells are then matched to the closest RGB values of Minecraft textures by using scikit-learn's nearest neighbor model, specifically the ball-tree algorithm [3]. This algorithm works well for matching 3D data as it allows for quick neighbor searching by splitting the space into hierarchical spheres. This is done by setting the two furthest points (Minecraft RGB values) and using them as pivots, assigning the other points to the closest pivot sphere. This is repeated for each subgroup until the group becomes less than or equal to the max group size. This creates a sort of algorithmic tree for the new point (cell RGB value) to traverse efficiently, having a time complexity of O(nlog(n)) for initial construction and O(n) for query.

Once the cells are mapped to Minecraft blocks, they are then placed into a 3D coordinate system based on their (x,y,z) values determined by the image coordinates and depth of the cluster. An adaptive z-value is used to determine how many blocks of depth each cluster should have. It takes the difference between the minimum and maximum depth of each cluster, and divides it by the max z-value given by the user. The thickness of each cluster is adjusted accordingly so there aren't any gaps in the build. This allows for accurate scaling of depth within the build. Using this information, the Litematica file can be made, a file that uses the Litematica mod in Minecraft to automatically place every block.

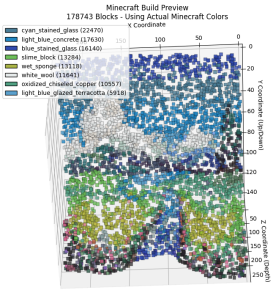


Figure 6. 3D coordinate system with Minecraft blocks mapped to each cell position.

IV. Experimental Results and Discussion

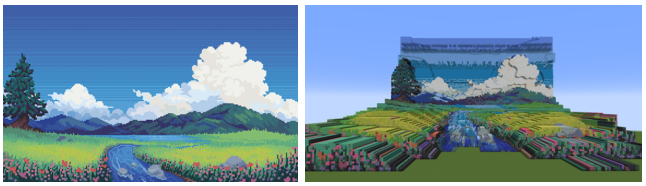


Figure 7. Mountain plains image converted to Minecraft.



Figure 8. Mizzou columns image converted to Minecraft.



Figure 9. POV driving image converted to Minecraft.

Shown in figure 7-9, the results are quite accurate, albeit not perfect, it is plain to tell that the Minecraft builds come from the test images used. There are several additions that could improve this pipeline and lead to better results.

An improvement that could be made is automatic scaling for converting it to Minecraft. Currently, the x and y components of the builds are scaled to 0.2, a value that through tests and trials that were found to produce good results for creating an accurate representation in Minecraft, an issue that could occur with this is with larger images, as in Minecraft there is a build limit in which the y-value can only be so large. With the addition of automatic scaling, it

could scale the build based on those limitations without altering the accuracy of the image, as part of the build would get cut off if that were to happen in its current state. In addition to automatic x and y scaling, having an option to automatically compute the max z-value based on the depth map would be very useful, eliminating user guessing for this value, though it is helpful for users with strict depth criteria.

Optimizing each function for better run-time would also be very beneficial, as depending on resolution of the image, cell size used, etc, the processing time for each run of this pipeline can be exceptionally long, with some tests taking longer than 30+ minutes. This would be a major issue if it were commercialized as users would spend hours trying to use photos with good quality, or would have to sacrifice quality by increasing cell sizes for faster run-times.

Depending on the image, certain minor issues can occur with the quality of the output build. For example, in figure 9, some of the small details in the car cause a sort of grainy effect on the build and in the more severe cases cause small holes to form. Better case handling for smaller clusters would help images with more detail feel more cohesive when converting to Minecraft. Another issue seen in figure 8, the textures on the sides of the columns receive some of the texture from the building, making it much less realistic. Making sure that each cluster and its thickness assume the same texture could fix this glaring issue, though that could lead to further problems.

Despite the many improvements that could be made to the pipeline, the results turned out quite successful. The builds accurately depict the input images used at smaller cell sizes and can act as 3D artwork in Minecraft builds. This could be used in many creative applications for builders. Future work for expanding this idea/pipeline is to implement this for video, using video tracking and SfM for larger scale and more detailed 3D reconstruction within Minecraft.

References

[1] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," Aug. 25, 2020. https://arxiv.org/pdf/1907.01341

[2] A. Mur, R. Dormido, N. Duro, S. Dormido-Canto, and J. Vega, "Determination of the optimal number of clusters using a spectral clustering optimization," *Expert Systems with Applications*, vol. 65, pp. 304–314, Dec. 2016, doi: https://doi.org/10.1016/j.eswa.2016.08.059.

[3] S. Omohundro, "Five Balltree Construction Algorithms," Sep. 20, 1989. https://www.academia.edu/70430197/Five_Balltree_Construction_Algorithms